Real-Time Orbital Image Analysis Using Decision Forests, with a Deployment Onboard the IPEX Spacecraft

Alphan Altinok, David R. Thompson, Benjamin Bornstein, Steve A. Chien, and Joshua Doubleday
Jet Propulsion Laboratory, California Institute of Technology, Pasadena, CA 91106
John Bellardo
Computer Science Department, California Polytechnic State University, San Luis Obispo, CA 93407

Received 5 December 2014; accepted 23 August 2015

Automatic cloud recognition promises significant improvements in Earth science remote sensing. At any time, more than half of Earth's surface is covered by clouds, obscuring images and atmospheric measurements. This is particularly problematic for CubeSats, a new generation of small, low-orbiting spacecraft with very limited communications bandwidth. Such spacecraft can use image analysis to autonomously select clear scenes for prioritized downlink. More agile spacecraft can also benefit from cloud screening by retargeting observations to cloud-free areas. This could significantly improve the science yield of instruments such as the Orbiting Carbon Observatory 3 mission. However, most existing cloud detection algorithms are not suitable for these applications, because they require calibrated and georectified spectral data, which is not typically available onboard. Here, we describe a statistical machine-learning method for real-time autonomous scene interpretation using a visible camera with no radiometric calibration. A random forest classifies cloud and clear pixels based on local patterns of image texture. We report on experimental evaluation of images from the International Space Station (ISS) and present results from a deployment onboard the IPEX spacecraft. This demonstrates actual execution in flight and provides some preliminary lessons learned about operational use. It is a rare example of a machine-learning system deployed to an autonomous spacecraft. To our knowledge, it is also the first instance of significant artificial intelligence deployed on board a CubeSat and the first ever deployment of visible image-based cloud screening onboard any operational spacecraft.

1. INTRODUCTION

Earth observation is a fundamental goal for orbiting spacecraft, and clouds are a fundamental challenge to this activity. Condensed water/ice clouds generally cover more than half of the Earth's surface (Mercury et al., 2012; Eastman et al., 2011; King et al., 2013). Traditionally, Earth-observing satellite missions accept clouds as an unavoidable contaminant. Operators schedule spacecraft observations in advance, downlink the resulting data, and filter cloud-contaminated measurements afterward. This means that about half of the downlinked data is not usable for its intended purpose. Operators have accepted cloud-related data loss as a necessary price of observing from space.

New spacecraft systems and designs may change this dynamic. New hardware is improving the computing power of Earth-orbiting spacecraft, making it possible to screen clouds onboard in real time. A spacecraft capable of detecting clouds could make real-time observation and downlink decisions to favor cloud-free areas. Just as the capability is becoming available, the need is becoming more acute. A new generation of small spacecraft (CubeSats) suffers much stricter bandwidth limits than larger missions, demanding that operators get the most of every downlinked bit. At the other extreme of the spacecraft design space, a new class of larger platforms can change attitude fast enough to dynamically target cloud-free areas. This includes "agile" spacecraft with control moment gyroscopes (CMGs) (Wie et al., 2002) and WorldView3 (DigitalGlobe, 2014). It also encompasses actuated instruments on larger platforms, such as payloads onboard the International Space Station (ISS). The current work is motivated especially by the OCO-3 instrument slated for use on the ISS (Eldering et al., 2013). OCO-3 is a downward-looking, atmospheric-sounding spectrometer with a very narrow field of view. It cannot see through clouds, but it is actuated and capable of dynamic retargeting, and its context cameras can seek out cloud-free regions just ahead. The stage is set for onboard cloud detection to enable selective downlink (for small spacecraft) and selective targeting (for agile spacecraft), significantly increasing mission yield at no additional cost in mass or bandwidth.

Most state-of-the-art cloud recognition algorithms are unsuitable for onboard use. Most require calibrated spectral information (Taylor et al., 2012) or a catalog of georectified reference images (Zhu and Woodcock, 2014), neither of which are available in real time. Several field

Direct correspondence to: Alphan Altinok, e-mail: alphan.altinok@jpl.nasa.gov

demonstrations have deployed simpler threshold-based methods: spectral cloud detection by EO-1 (Griffin et al., 2003) and Bayesian spectral cloud screening onboard the AVIRIS-NG instrument (Thompson et al., 2014a). However, these deployments rely on short wave infrared (SWIR) spectral information that would not be available to most spacecraft.

This research applies a more traditional computer vision solution. We use statistical machine learning to classify pixels based on local texture observed by an uncalibrated visible wavelength camera. A random forest evaluates local pixel intensity patterns to assign a classification to each pixel independently. The resulting cloud mask can then inform instrument targeting or downlink decisions. We describe two different experiments. First, we evaluate image sequences from the ISS high definition video (HDEV) system (Runco, 2014). This scenario, analogous to an OCO-3 deployment, evaluates the classification performance for nadir-pointed imaging. Second, we present results from a deployment of the image-based cloud screening onboard the IPEX CubeSat. This demonstrates actual execution in flight and provides some preliminary lessons learned about operational use. It is a rare example of a machine-learning system deployed to an autonomous spacecraft. To our knowledge, it is also the first instance of significant artificial intelligence deployed onboard a CubeSat and the first ever deployment of visible image-based cloud screening onboard any operational spacecraft.

2. PRIOR WORK

Most cloud screening methods are intended for science data "retrieval algorithms," analyses that estimate physical properties of the surface. Here, cloud screening acts as a preprocessing step that prevents contamination of downstream maps and statistics. Algorithms vary widely in their methods and complexity. "Classical" cloud screening is based on simple thresholds on spatial and spectral properties of the image (Wang and Shi, 2006). Perhaps the most widely used is the MODIS algorithm, which compares selected visible and near-infrared (VNIR) and near-infrared (NIR) bands to thresholds and then combines these results in different combinations, depending on the surface type (Ackerman et al., 1998; Frey et al., 2008; Ackerman et al., 2008). The MODIS algorithm involves dozens of independent threshold tests motivated by different spectral trends and features, underscoring the challenges of building reliable cloud screening using spectral properties alone. Other algorithms estimate atmospheric properties from absorption features such as the oxygen A band (Gómez-Chova et al., 2007; Taylor et al., 2012). Minnis et al. predict clear sky brightness temperature values in thermal channels, using information from ambient temperature and humidity (Minnis et al., 2008). These NIR and thermal infrared measurements require dedicated, carefully calibrated instrumentation. Another class of algorithms involves comparisons against historical images. Recent work by Zhu and Woodcock exemplifies the state of the art in this domain (Zhu and Woodcock, 2014). These approaches require accurate radiometric calibration and georectification of new images, so they are also unsuitable for onboard use.

This work focuses entirely on spatial structure in single visible-wavelength images. This is suitable for a wide range of sensors such as the OCO-3 context cameras. Framing cameras can have larger fields of view, providing enough prior warning to enable adaptive instrument targeting on the fly. Unsurprisingly, there are also many previous examples of cloud detection in framing images. Han et al. use a simple intensity threshold to detect thick clouds and then use a second step to eliminate thin clouds with the help of a landmark-based coregistration against historical images (Han et al., 2014). Liu et al. use a superpixel segmentation to identify homogeneous areas and to apply a separate threshold to each (Liu et al., 2015). Zhang et al. create screen airborne images by combining color thresholding with a bilateral filter in the image domain (Zhang and Xiao, 2014). A few studies have attempted to incorporate local texture into the classification decisions. Martins et al. show that the standard deviation of reflectances in a 3×3 pixel window can be used to discriminate clouds from aerosol plumes over the ocean (Martins et al., 2002). Other work has used spatial information as a smoothing criterion, such as Murtagh et al. that represent spatial dependencies using a probabilistic Markov random field (MRF) prior (Murtagh et al., 2003). Of special relevance to this work, an early study by Lee demonstrated neural networks to recognize clouds by their high-spatial heterogeneity (Lee et al., 1990). The all-sky imager (Cazorla et al., 2008) uses a neural network trained to recognize clouds in upward-looking visible panoramic color images.

In contrast, onboard cloud screening is very rare. It was demonstrated in early experiments onboard the EO-1 spacecraft (Griffin et al., 2003). This algorithm relied on infrared spectral information, normalizing radiance measurements for solar input to yield an apparent top of atmosphere (TOA) reflectance. The EO-1 algorithm then applied a branching sequence of threshold tests based on carefully crafted spectral ratios. This distinguished clouds from other bright features such as snow and desert sand. More recently, cloud screening was demonstrated in real time onboard the AVIRIS-NG instrument that was deployed on a Twin Otter research aircraft (Thompson et al., 2014a). This system used nonparametric density estimation with a Bayesian formalism to set optimal thresholds on two or more spectral channels. Both of these deployments relied only on spectral information, classifying pixels independently and ignoring spatial structure. This greatly simplified the pattern recognition problem.

The brightness thresholding methods used in prior onboard methods are generally not feasible for a visible

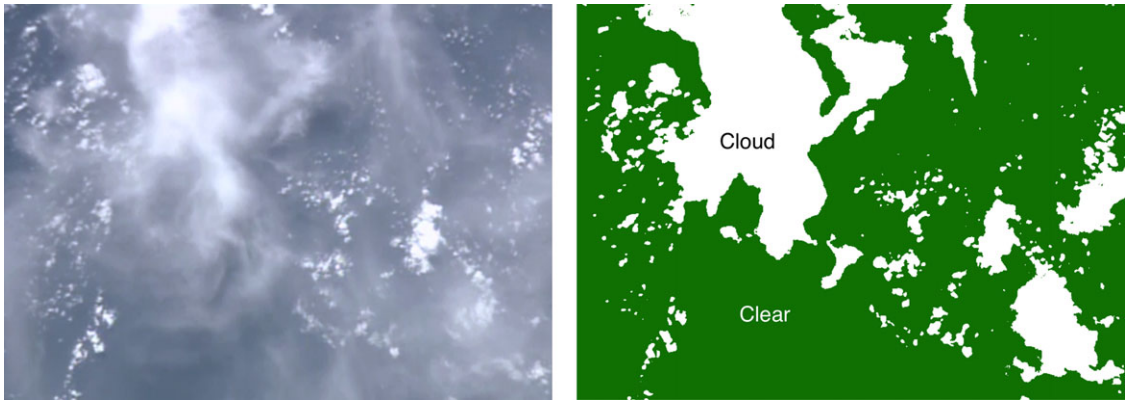

Figure 1. A frame from the nadir-pointed HDEV camera onboard the ISS (left), and the result of Otsu's thresholding method (right). Only denser clouds are highlighted, leaving a majority of the frame as clear, where in fact most of the frame is covered with clouds.

monochrome or RGB framing camera. In visible wavelengths, the raw brightness values of individual pixels are insufficient discriminators. It is natural to look for other threshold-based alternatives, such as classical Otsu thresholding (Otsu, 1979), that could find a new threshold appropriate to the contrast of each scene. However, such methods require cloud and terrain intensity levels to be well separable and fare poorly when clouds show a range of brightnesses. In addition, determining a suitable threshold level requires that both cloud and terrain pixels are well represented. Figure 1 shows a counterexample where nearly the entire frame is covered with clouds at varying intensity levels, leading to a poor classification result.

This suggests that for uncalibrated RGB images, pixelwise analyses are insufficient and some kind of spatial analysis is necessary. Unfortunately, many spatial techniques are too computationally complex for onboard use. Spacecraft processors are highly power-constrained, often lagging the commercial state of the art by a decade or more. Total processor speeds well under 100 MIPS are common, and only a small portion of the duty cycle would be devoted to image analysis. Furthermore, many spacecraft processors (including the main IPEX processor in our flight demonstration) lack hardware floating point support. This precludes a large number of popular segmentation methods, such as the iterative solutions to MRF. Lack of floating point precludes a wide range of template classifiers such as support vector machines (SVM). While such methods are more robust to intensity variations and are reasonable contenders for deployment neither is a panacea and would likely require algorithmic modifications for fixed-point execution.

Given the large effort already devoted to maturing ground-based pipelines, it is not our goal to invent a more accurate cloud screening algorithm. Instead, we aim to demonstrate a system that can produce meaningful improvements vis-à-vis blind targeting while operating under the strict computational constraints of modern spacecraft. We explore the operational implications of deploying cloud screening on an orbital system and using these methods onboard to inform robotic data collection and instrument decision making. For the first time, we deploy onboard image understanding, including cloud detection to replicate these results in space. To this end, our cloud screening algorithm is conservative - a simple random forest based approach that has already shown promise in space exploration domains (Wagstaff et al., 2013; Bekker et al., 2014) and is straightforward to adapt to spaceflight processors. However, the application to flight is new. Finally, we provide an assessment of the potential improvements in data yield that can be achieved by this or similar real-time cloud screening methods.

3. ALGORITHMIC APPROACH

We desire an algorithm that is simple to implement, fast to execute, with reasonable accuracy and robustness, and can use input from a visible camera. Our approach uses the random forest pixel classification of Shotton et al. (Shotton et al., 2008). This method has recently been applied to perform pixel classification in a wide range of space robotics problems, such as geologic surface analysis in Mars Rover images (Thompson et al., 2012; Wagstaff et al., 2013); planetary analogue rovers and instruments (Foil et al., 2013; Wettergreen et al., 2014); and recently, comets and asteroids (Fuchs et al., 2014). Not surprisingly, the basic approach serves to classify surface features and clouds from Earth's orbit. Diverse implementations are available, including the TextureCam codebase (Thompson et al., 2014b) and FPGA-based systems recently demonstrated on flight-relevant hardware (Bekker et al., 2014).

The random forest is an ensemble of bagged decision trees that classifies each pixel independently (Breiman,

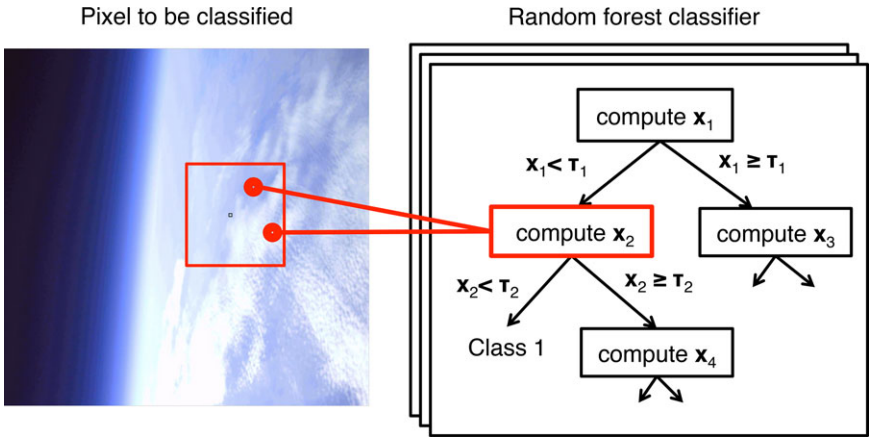

Figure 2. We calculate visual features using arithmetic operations applied to a pair of pixels in the local context window around the target pixel.

2001). It models the probability $P(Y|X)$ representing the probability of class Y given the attributes X of the local image neighborhood. We learn this relationship from a training set of labeled pixels. Each pixel p is associated with an attribute vector $x \in \mathbb{R}^n$ and a class $y \in \mathbb{N}$. In our case, attributes are typically simple arithmetic operations applied to pixels' RGB color values at specific locations relative to the pixel being classified (Figure 2). Each node i is associated with a specific attribute $x_i \in X$ and a threshold τ_i. Upon reaching the node, a pixel's attribute is calculated and compared to the threshold to decide whether the control should descend left or right. These halves correspond to different subpopulations on either side of an axis-parallel binary partitioning of the input space. Each leaf node contains a posterior class distribution learned from training data; these probabilities are averaged to form the final classification result.

We train the model with manual labels of representative images. The training procedure grows the tree from the root node, expanding each leaf in turn until a maximum depth is reached or the population of pixels reaching the child node is homogeneous. Following the approach of Breiman (2001), we train each random forest on a separate subset of the data. To grow a node, we search over a random selection of potential attributes, calculating the threshold that best reduces the expected posterior entropy of the resulting left and right populations (L and R). The expected entropy is defined as follows:

$$E[H(L,R)] = \frac{|L|}{|L \cup R|} \sum_y P(y|x_i < \tau_i) \log P(y|x_i < \tau_i) + \frac{|R|}{|L \cup R|} \sum_y P(y|x_i > \tau_i) \log P(y|x_i > \tau_i) \quad (1)$$

Valid attributes include pixel sums, differences, raw values, and ratios. Free parameters of the training process include the number of trees, the number of expansions per tree, and the width of the context window. We considered different color spaces and preprocessing strategies, including denoising and frequency-space filtering operations. However, none significantly exceeded the performance of the raw RGB values. Consequently, we used the raw frames for operational simplicity.

4. ISS EXPERIMENTAL METHOD AND RESULTS

This section presents a series of experiments to evaluate onboard cloud screening for a self-targeting instrument in low earth orbit (LEO). We focus our case study specifically on the ISS, an orbiting space platform and a likely host for gimbaled, targetable science instruments such as the Orbiting Carbon Observatory 3 (OCO-3) (Eldering et al., 2013). Because a large library of ISS images are available, we use these tests to explore the performance profile of different algorithm options in a controlled experimental setting with large sample sizes.

OCO-3 is a compelling application example. This instrument will deliver the space-based measurements of atmospheric carbon dioxide (CO_2) with the precision, resolution, and coverage needed to improve understanding of surface CO_2 sources and sinks (fluxes) on regional scales ($>$ 1,000 km) and the processes controlling their variability over the seasonal cycle. It incorporates three high-resolution grating spectrometers designed to measure the NIF absorption of reflected sunlight by CO_2 and molecular oxygen (O_2). OCO-3 incorporates an agile two-axis (azimuth and elevation) pointing mechanism on the nadir instrument deck in order to make targeted measurements over land and ocean during all sunlit hours. To validate the pointing system and to simplify ground geolocation and pointing knowledge reconstruction, two commercial color context cameras are

mounted with the pointing mechanism. The first camera is co-boresighted with the instrument, but the nature of the optical path makes the image effectively monochromatic. The second color camera is attached to the outer housing of the pointing mechanism. The experiments in this section use monochrome images, based on the premise that only the instrument-aligned camera data will be available for onboard analysis.

OCO-3's nominal observation schedule will maintain the instrument at nadir over land and actuate to point off-track over ocean (the specular sun glint off the water at selected angles provides improved SNR over dark surfaces). In the proposed operating mode, camera information could be used in real time to autonomously point the sensor and to target cloud-free locations. This section quantifies the possible improvements from random forest based cloud screening, measured by the total yield of cloud-free soundings.

Our tests used imagery from the ISS High Definition Earth Viewing experiment (HDEV), a suite of onboard cameras providing a live stream of high-definition Earth images through four commercial high-definition cameras (Runco, 2014).

The HDEV experiment had multiple cameras fixed in separate nadir- and horizon-pointing directions. It cycled regularly between these views, transmitting the live video feed directly to Earth. We favored the nadir-pointed images for similarity to the OCO-3 context cameras. We randomly sampled videos from nadir-pointing segments of previously recorded HDEV experiments. The HDEV data were encoded for distribution on Earth, and some individual frames acquired from these videos showed slight degradation in the form of blurry edges around cloudy areas. However, the classifiers seemed robust to these artifacts, and we did not notice any significant performance penalty from these artifacts.

4.1. Training

We evaluated several different algorithm options. The first was a simple adaptive thresholding approach, which would be suitable for spaceflight deployments but that we anticipated would perform poorly. The second was a support vector machine (SVM) classifier, representing a state-of-the-art classifier that was not suitable for flight due to floating point requirements. The third algorithm was the proposed random forest approach, which we hoped would provide adequate performance while also satisfying hardware constraints. We sampled 512 frames from representative image sequences (Ustream, 2014) that show significantly different cloud patterns, cloud coverage, and ground features. The sequences depicted a large variety of scenes, ranging from land or ocean regions covered by thick, spotty cloud patches, to translucent clouds covering entire frames, to mostly cloud-covered areas that varied in thickness. We reserved 12 frames with manually labeled cloudy and background areas to train the SVM and random forest classifiers. The remaining 500 frames were used for testing.

To preserve similarity with the monochrome OCO-3 camera, the classifier input was a single channel - a grayscale image obtained from the RGB representation, similar to a panchromatic imager. We exhaustively hand-labeled 12 representative frames with various cloud densities and coverage and bifurcated them into training and test sets. We used a 5 x 5 neighborhood of raw grayscale intensities to train both algorithms.

During training, we found the random forest performance was insensitive to model parameters such as the window size and number of nodes. This suggested that the underlying pattern problem was not very complex; an effective decision was possible after testing only a few neighbors close to the classified pixel, and the most important decisions could be made at the higher-order nodes. In training SVMs, we experimented with using linear and RBF kernels and found better performance from the latter. For RBF kernels, out-of-sample misclassification estimated from 10-fold cross validation was 4%, compared to 22% for linear kernels. Both SVM and random forest methods showed adequate accuracy for cloud detection. However, SVMs with RBF kernels require significantly more onboard computational resources, and in practice would demand hardware support for floating point operations.

4.2. Evaluation

The distinction between cloudy and clear pixels was subjective. Occasionally, the boundary edges showed a slow, continuous transition between cloud and clear sky. To avoid ambiguity, we focused on the thickest, unambiguous clouds. We considered a pixel to be cloudy when it was opaque, for example, thick enough to obscure the ground texture beneath. A pixel was clear only if it showed no brightening, blurring, or other visible evidence of any cloud contamination. Any ambiguous pixels that did not obviously fall into one of these categories were excluded from the performance analysis. We evaluate performance using two scores: the raw classification performance, defined as the fraction of pixels that would be classified correctly; and a task-oriented evaluation, defined as the improvement in science data yield of the targeted instrument. Figure 3 shows representative frames, together with the autonomous classification result.

We compared performance of random forests, SVMs, MRFs, and Otsu's thresholding. Both random forests and SVMs used the entire local neighborhood of pixel intensities as input. Figure 4 shows a typical example. In general, the two classifiers produced very similar results. Otsu's thresholding and MRFs also gave similar output but failed to detect many cloudy areas. Because MRFs required

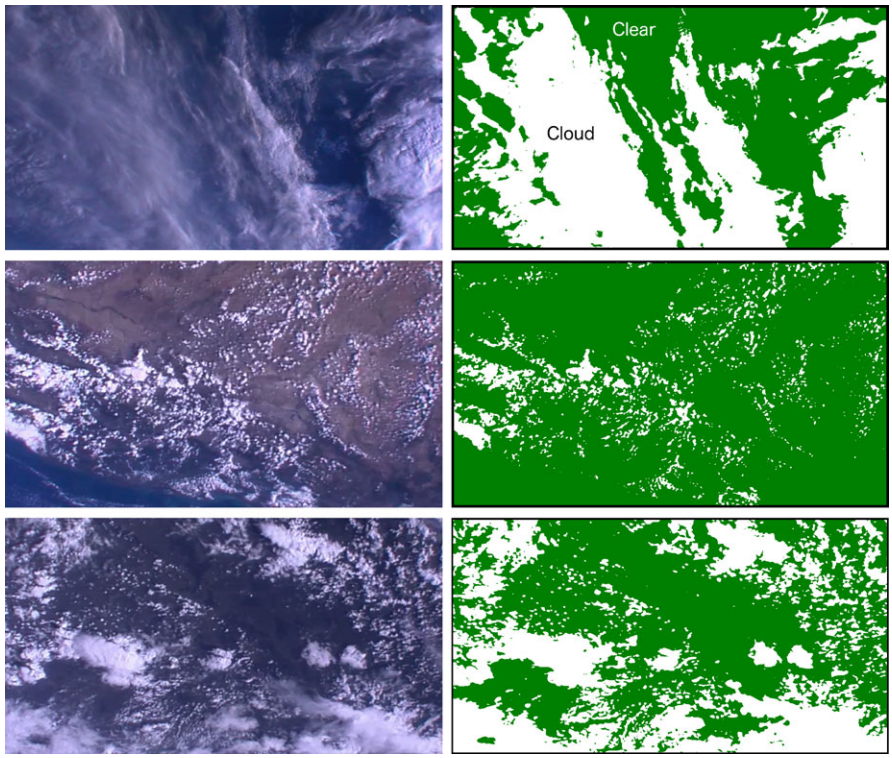

Figure 3. Typical scenes from the nadir-pointed HDEV camera onboard the ISS. Top row: ocean covered by a mixture of translucent and thick clouds. Middle row: land covered by spotty patches. Bottom row: ocean covered by mixed cloud patches.

infeasibly long runtimes without any clear accuracy or robustness advantage, we did not consider them for further evaluation.

For the remaining methods, we quantified the raw pixel-wise classification performance. Figure 5 shows classification performance as a receiver operating characteristic (ROC) curve, comparing the random forest approach with a static brightness threshold that we adjusted across the [0,255] interval and the results of the SVM approach. All three methods perform within a false positive rate of less than 0.05; both SVM and random forest methods were more robust to intensity and coverage variations than simple thresholding.

The SVM appears to provide the best overall performance. However, the ambiguous cloud pixels can confuse fine distinctions between such high levels of accuracy. We conclude that both methods provide adequately robust alternatives to simple thresholding, which is corroborated by a visual inspection of the results.

We next sought to quantify improvements in mission science yield. The combined data set for training and testing amounted to more than 22 million data points in 500-plus frames. We identified target points that were farthest from any cloud using a "largest inscribed circle" approach. This method has become commonplace for spacecraft instrument targeting (Estlin et al., 2012). We thresholded the posterior cloud probabilities to form a binary image and applied the distance transform to identify pixels that were far from the cloud boundaries. Figure 6 shows the result of the distance transform applied to target selection. While we feared the distance transform would be sensitive to single-pixel noise, we found this was not a problem because the random forest output already incorporated some amount of local spatial smoothing.

The accuracy provided by the random forest significantly improved the quality of the resulting targets vis-à-vis simpler thresholding. Figure 7 shows the boundaries produced by Otsu's method, compared with the boundaries produced by the random forest. Note that Otsu's method finds a threshold by minimizing within-class variances for background and foreground classes. This assumed both classes were well represented, which was often false. The distance transform from the random forest showed the best overall performance; it nearly always selected targets lying in the center of open areas.

We manually evaluated the accuracy of the selected target location using three alternatives: the *status quo* strategy for pointing in a nadir-looking position; a global intensity threshold, set by held-out training images to produce the best result and then applied universally to all sequences;

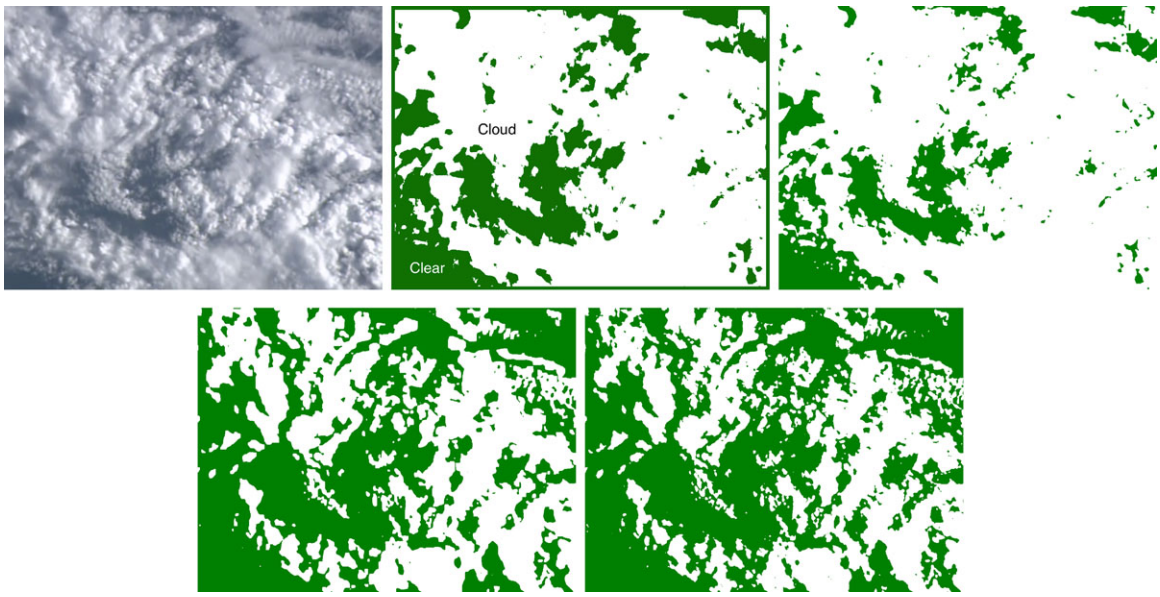

Figure 4. Visual comparison between alternative methods considered. Top row, left to bottom right: Original frame, random forest, SVM. Bottom row, left to right: MRF and Otsu's method.

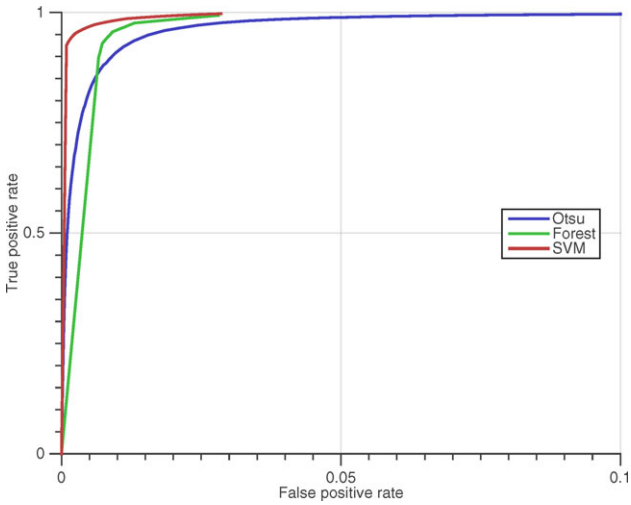

Figure 5. ROC curves showing comparison of forest, threshold, and SVM methods. Each method performs within a false positive rate of 0.05.

and, finally, the proposed random forest method, which adaptively accounted for the local intensity patterns in each pixel's context window. Table I shows the number of images from each sequence in which clear sky was targeted. The failure rate for the random forest was extremely low over land but rose slightly over ocean because many images were completely clouded over. In these congested scenes, it was common that a small wisp of cloud subtending just a few pixels would lie on the target point. According to the most conservative analyses, over half of the Earth's surface is covered by clouds (Mercury et al., 2012; Eastman et al., 2011; King et al., 2013). This suggested that up to a doubling of science data yield over land would be possible, which is consistent with the improvement shown here. We note that support vector machines produce very comparable results to random forests. Their notable disadvantage was on cloud configurations where over detection (see Figure 4) caused targeting to skew slightly.

In summary, for classification performance and for targeting, both random forests and SVMs provided comparable results. The SVM accuracy was superior, though it would not be feasible for systems without floating point hardware. Both classifiers represent a significant improvement over blind targeting or simple intensity thresholding. Otsu's method improves the performance of thresholding; yet, it operates on individual frames and thus depends on cloud and land coverage to be comparable.

Note that for this demonstration we chose to classify every pixel in each frame. This might be too computationally intensive for some spacecraft processors, even for the speedy random forest. However, the runtime could be made much faster - even trivially fast - simply by classifying a smaller number of candidate target points in each image. A course grid of candidate target locations could be evaluated in subsecond time on most spaceflight avionics systems. The recent implementation of the random forest algorithm to the spaceflight-relevant Virtex V Field Programmable Gate Array (FPGA) (Bekker et al., 2014) is another compelling solution.

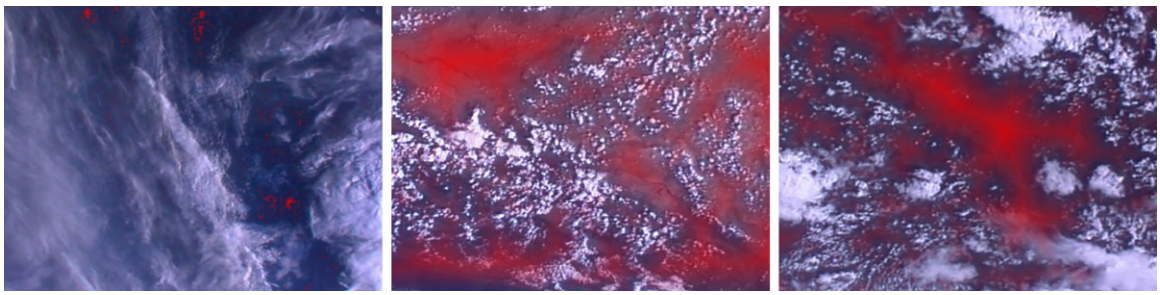

Figure 6. Results of distance transform (in red) superimposed on the ISS images. Higher intensities indicate better targeting to avoid clouds.

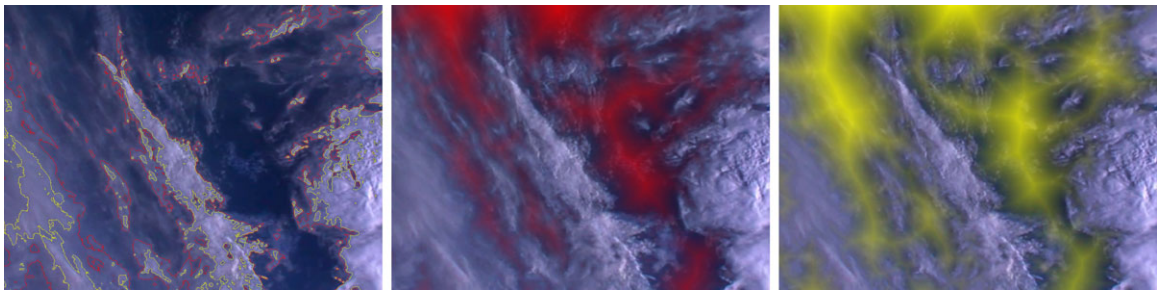

Figure 7. Left: Red outline shows the result of our method. The yellow outline shows the result of Otsu's method. Distance transforms that indicate best targeting positions based on cloud regions detected by the decision forest (middle) and by Otsu's method (right).

Table I. Targeting accuracy for cloud-free image areas.

Sequence	*Status Quo*	Global Threshold	Random Forest
Sequence A, Land (n=68)	30 (44.2%)	62 (91.2%)	68 (100%)
Sequence B, Ocean (n=70)	13 (18.6%)	23 (32.9%)	37 (52.9%)
Sequence C, Water Features (n=40)	22 (55.0%)	29 (72.5%)	36 (90.0%)
Total (n=178)	65 (36.5%)	114 (64%)	141 (79.2%)

5. DEMONSTRATIONS ONBOARD THE IPEX SPACECRAFT

This section describes a deployment onboard the Intelligent Payload Experiment (IPEX), a spacecraft that has operated on orbit since December 2013 (Figure 8). The primary mission goal was to validate autonomous operations for onboard instrument processing and product generation for the Intelligent Payload Module (IPM) (Chien et al., 2009, 2010, 2013) of the proposed Hyperspectral Infra-red Imager (HyspIRI) mission concept, on behalf of NASA's Earth Science Technologies Office (ESTO). Future missions such as HyspIRI will produce enormous volumes of data requiring either significant communication advancements or data reduction techniques. IPEX demonstrated several technologies for data reduction that use onboard image analysis and general operations autonomy (Chien et al., 2014). Here, we describe a demonstration of real-time cloud detection.

IPEX is a CubeSat: small, standardized spacecraft design intended to be deployed from the upper stage of a much larger mothership (Swartwout, 2014). During launch, it is isolated within a deployment mechanism, a spring-loaded box that encapsulates the smaller spacecraft until release. This isolation means the mothership and CubeSat can be developed virtually independently, providing a true "piggyback" launch capability. Since its introduction in 1999, the standard has provided hundreds of launches for industry, commercial, government, and university groups. Such missions rely heavily on commercial-off-the-shelf (COTS) components and lightweight testing, verification, and assembly to reduce costs. Consequently, failure rates are high: less than 25% of CubeSat missions to date achieve their primary success criterion (typically defined as a simple on-orbit activity such as transmitting sensor data) (Swartwout, 2014). Most successful missions end soon after checkout due to unstable orbits, component failure, or other unknown

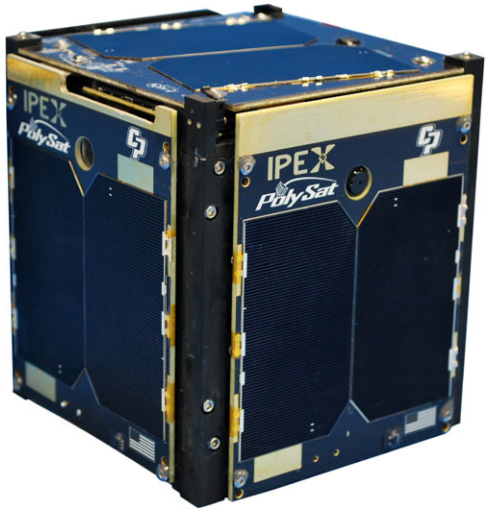

Figure 8. The IPEX CubeSat.

causes. Despite the high failure rate, CubeSats have become an increasingly popular platform for simpler government and commercial missions. As of this writing, more than 200 such CubeSat missions have attempted launch (Swartwout, 2012). However, to our knowledge, there has been no previous attempt to deploy artificial intelligence, computer vision, or pattern recognition onboard a small spacecraft platform.

6. IPEX HARDWARE AND SOFTWARE ARCHITECTURE

IPEX was a 1 unit (1 U) CubeSat, approximately 10 cm x 10 cm x 10 cm in size (Figure 8). All six faces of the spacecraft had solar panels generating 1.0 – 1.5 W when illuminated. The spacecraft carried three batteries to enable operations during eclipse and continuous processing modes. Five Omnivision OV3642 cameras were arranged on five sides of the spacecraft to improve the odds of observing Earth from any orientation. In theory, the passive magnetic attitude control would stabilize the spacecraft orientation, but it proved challenging to reliably model this effect. In practice, any one of the cameras could be in view of Earth. Each camera was 3 megapixels at maximum resolution, with each pixel having a finest instantaneous field of view of 0.024 degrees. The IPEX orbit ranged as low as 400 km, where these cameras enabled approximately 200 m/pixel imagery of the Earth's surface at nadir.

The spacecraft carried two main computers. The main Command and Data Handling (C&DH) system was a 400 MHz Atmel ARM9 CPU without hardware floating point (Figure 9 Left). Storage consisted of 128 MB RAM, 512 MB NAND, and a 16 GB micro SD card. The flight computer communicated directly with the onboard radio, a custom communications system operating in an amateur radio frequency of 437 MHz with 1 W total power output that provided a maximum data rate of 9.6 Kbps. IPEX also carried a Gumstix Earth Storm computer-on-module, which included an 800 MHz ARM processor, 512 MB RAM, 512 MB NAND flash, a 16 GB micro SD card, and a Linux operating system (Figure 9, center). The Gumstix required less than 1 W during operation. It was controlled through several layers of fault protection on the C&DH board. Communication between the C&DH and Gumstix boards used a 200 Kbaud serial line.

Figure 10 shows the software system architecture. The flight software on IPEX was based on extensions to the Linux operating system, with multiple tiers of fault protection providing robustness to radiation-induced upsets, bit flips, and latch-ups. The primary line of defense was a low-level health monitor process known as the "system manager." The system manager periodically evaluated the health status of other onboard processes and sensors. After verifying that all subsystems were nominal, it sent a regular heartbeat to a hardware watchdog, which would otherwise reset the spacecraft. The system manager was also responsible for monitoring critical hardware and satisfying basic mission-critical constraints (for example, switching off the Gumstix co-processor if the battery power fell too low). The telemetry process was a separate module that ran simultaneously on the main Atmel computer. It periodically read the real-time state from the system manager and archived it for later downlink. Next, the sequence execution process

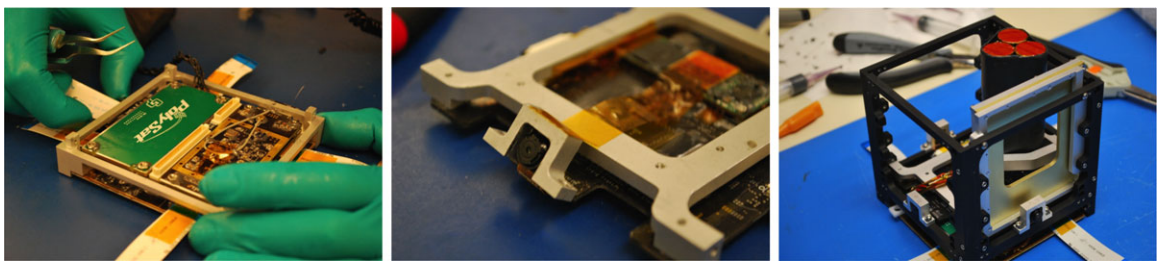

Figure 9. IPEX avionics system. Left: Calpoly intrepid board containing radio, the Atmel flight processor, and antennas. Center: The gumstix coprocessor (at center). Cameras are mounted to the frame with aluminum brackets. Right: Installation in the *1_U frame, showing the position of batteries.*

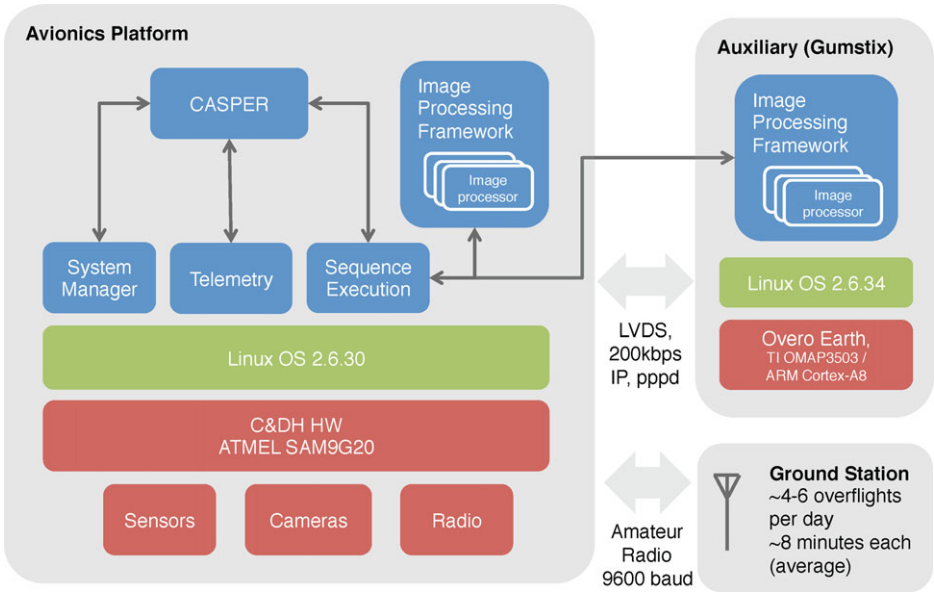

Figure 10. IPEX software and hardware system architecture.

(Datalogger SeQuence Execution Engine, or DSQEE) was the topmost interface to the planning system. It executed sequences of commands, ensuring child processing in the proper order, enforcing resource limits on the child processes, and providing basic fault protection in the form of simple OS calls and checking of exit codes. The execution engine provided options for real-time, time-based, and event-based commanding. The image processing framework was a separate bundle of software for image data processing, a separate copy of which was maintained on both computers. It included image processing routines and wrapper scripts, as well as some standard C image processing executables from the libpgm library and a handful of Linux command line utilities. The main autonomy component, CASPER, is described in the following section.

6.1. IPEX Operations and Planning

IPEX serviced observation requests from multiple sources, including humans or automated process on the ground as well as closed loop automated processes onboard. Observation requests were handled in a priority-based fashion by the onboard CASPER planner (Chien et al., 2000). CASPER incorporated a model of the system, the observation request goals and activities to fulfill them, and constraints and resources such as CPU usage, RAM usage, the available data storage capacity, the battery state-of-charge, and power generation. It used this model to maintain an in-memory schedule of activities maintaining health and a consistent state through the projected future. CASPER also interfaced with the flight telemetry system to monitor the state of these resources in real time (Figure 10). When they deviated from the previously projected values enough to induce inconsistencies anywhere in the plan, CASPER would execute a mixture of generic and IPEX-specific functions to repair the schedule. This could involve moving, adding, or deleting activities and goals to reach a consistent state. CASPER would then work to fulfill any unscheduled goals (observation requests) with a priority inversion compared to the fulfilled goals.

Figure 11 illustrates a typical timeline. Modeled events include ground contacts and periods of solar illumination. Consumable resources, represented as blue histograms, include battery energy and data storage capacity. Activities include image processing by the Gumstix and Atmel computers and occasional downlink events. CASPER would track dependencies of each activity, along with dependencies on the available resources and the state of each subsystem (for example, whether the auxiliary Gumstix is powered). This timeline spans 1 week of actual operations.

More important to this demonstration, CASPER was also capable of responding to onboard analysis of instrument data such as detection of features or events in imagery. Onboard processing was used to triage low-value data products (e.g., images of dark space) early in processing; it could bypass further processing and prevent wasting limited resources. For images passing this gate, CASPER could schedule follow-on acquisitions based on event or feature detection. The CASPER model for IPEX represented a number of software processing workflows and operations constraints. The basic processing flow of an observation request was to first acquire imagery with a camera and

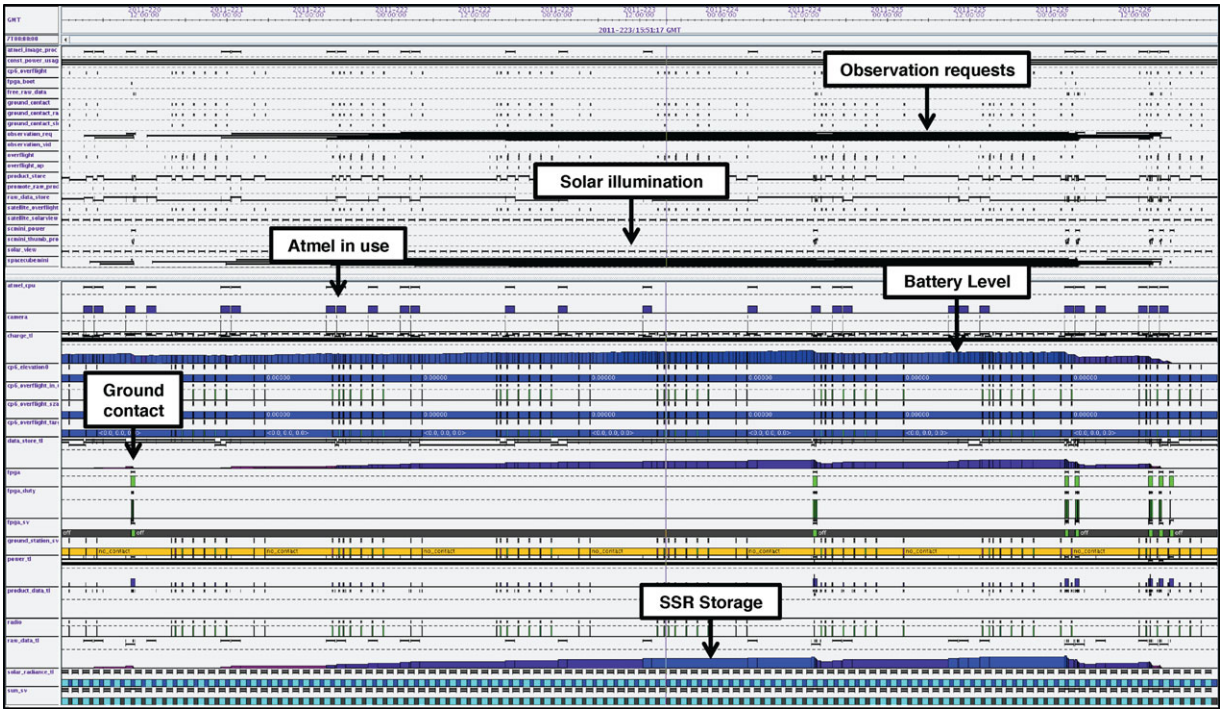

Figure 11. Typical CASPER timeline illustrating resources, activities, and events. This image is best viewed in an electronic copy at high resolution.

to then process the image with a preliminary assessment that scores the image as interesting. "Interesting" images were then processed on both Atmel and Gumstix processors, with the full range of selected image processing algorithms. The results were compared for any differences, and a summary statistic was inserted into the telemetry stream. Finally, the system assessed the resulting products for generating a follow-on goal that might result in future opportunistic acquisitions or processing. This sequence has already been repeated many thousands of times over the course of the IPEX mission.

Communications with IPEX used a dedicated ground system at CalPoly San Louis Obispo (Figure 12). This station had an articulated antenna frame with dual 437-MHz directional Yagi antennas. The spacecraft was typically above the horizon for approximately 12 minutes at a time, though signal levels varied due to distance and tracking accuracy; in practice, it never achieved an average transfer rate of 9.6 Kbps for the entire 12-minute window. There were approximately six communication passes per day. During a pass, the autonomous ground operations system would perform a number of activities. It would downlink engineering telemetry because the last ground contact, along with summary statistics on the onboard processing such as the metadata of acquired and processed images, algorithm runtimes, and comparisons of results. Finally, it downlinked a small subset of the images and/or products for ground validation. These were generally highly compressed thumbnails but occasionally included prioritized images and products or specific requested files.

In this manner, IPEX has been used to validate a wide range of onboard instrument processing algorithms. The vast majority were variations of pixel mathematics, such as normalized difference calculations or band ratios (Brakenridge and Anderson, 2006; Ip et al., 2006; Carroll et al., 2009). IPEX also demonstrated more computationally complex image processing technologies, including support vector machine classification techniques (Doggett et al., 2006), detection of salient image landmarks (Chien et al., 2014), and endmember detection (Thompson et al., 2013). It also validated techniques for atmospheric correction Tho (2008) by applying the FLAASH algorithm (Perkins et al., 2012) to preloaded spectral data.

6.2. IPEX Onboard Cloud Detection

We devised a demonstration random forest cloud classification using the IPEX onboard cameras. We first defined a small set of relevant classes that would be easiest to discriminate reliably from RGB images and that could be useful to selective downlink. We observed that cumulus clouds in the near-field had distinctly different appearance from the scattering at the horizon. We captured this difference by sorting cloud features into "haze" and "cloud" classes

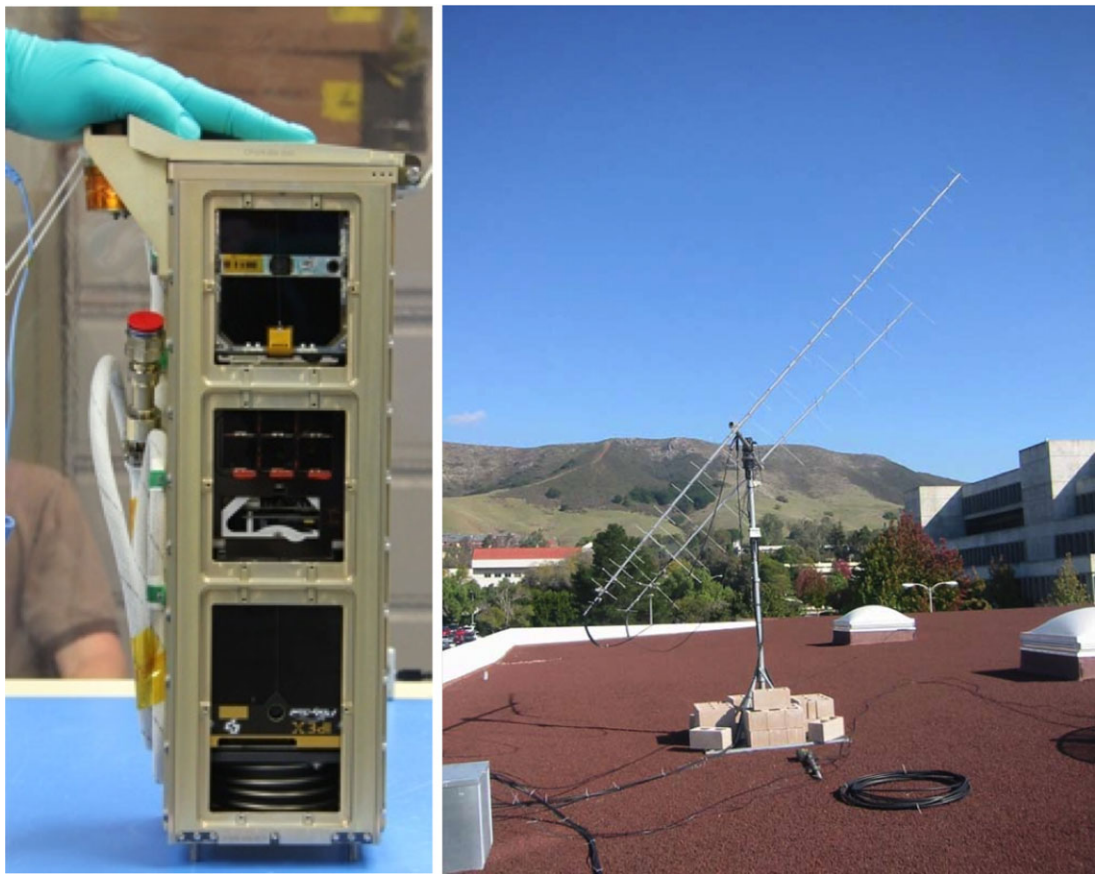

Figure 12. Left: IPEX integrated into the CubeSat deployment system. Right: the IPEX tracking station at San Louis Obispo.

for training. We also combined land and water into a common class due to paucity of training data to teach the distinction. In any case, the land/water distinction was not deemed particularly useful to cloud screening. In summary, the four classes were "Outer space," indicating dark sky; "Haze/limb," which labeled indistinct clouds as well as haze near the planetary horizon; "Cloud" for bright opaque cloud formations; and "Surface" for cloud-free ocean or land. The coverage fraction of these four classes formed a very concise and informative description of image content for downlink.

Naturally orbital images from the IPEX cameras were not available before launch. We pretrained the random forest model in advance using test images acquired by a prototype camera onboard a high-altitude balloon. The balloon exceeded 30-km altitude, flying above nearly all of the atmosphere, cloud, and Rayleigh scattering (Figure 13). This experiment used the same model camera as the spacecraft, so it offered the best available terrestrial analogue to the expected image content to be observed from orbit. Our tests suggested that performance plateaued after a small number of training frames. We labeled three representative nadir-pointed images and trained a random forest model with 16 trees and a local window of 41 pixels. Training performance was insensitive to these parameters; they were chosen to be well along the asymptote of the speed/performance curve, such that changes by a factor of 2 did little to enhance or degrade performance. We installed the pretrained model into the spacecraft prior to launch.

There were several operations constraints that would preclude the upload of a new random forest trained on orbital data. One was the intrinsic size of random forests relative to images. Downlinked images were highly compressed with JPG compression and were seldom more than 10 KB. In contrast, a parsimonious random forest file, now optimized such that performance started to degrade noticeably, was about 50 KB compressed. In addition, the uplink bandwidth was more constrained than downlink because the system had been originally designed to receive commands rather than data files. Uplinking a new classifier would have required separating the forest into fragments and uploading them over the course of many separate communications passes. This was deemed operationally infeasible because of the difficulties of getting 10 dedicated passes for this

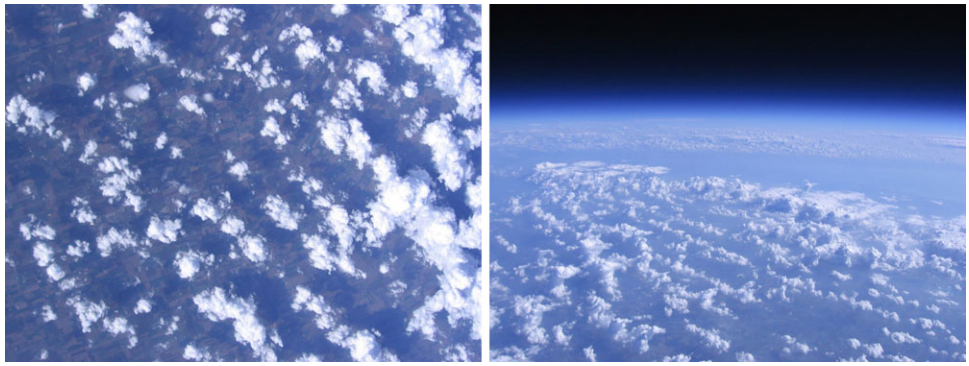

Figure 13. Nadir- and side-pointed training images acquired from the IPEX high-altitude balloon test using flight model cameras. Images were acquired at an altitude of 30 km.

purpose. Calculations suggested that, if all other system elements worked perfectly, uploading a new random forest could fully occupy the mission for up to a month, which was deemed infeasible for a short-lifetime mission. We note these constraints for completeness, though they would not be a factor for most missions, and it would be better to train a classifier during an initial mission checkout phase when realistic on-orbit images were available.

At runtime, the onboard classification would execute a simple procedure on each new image. First, it would subsample the frame to a 384×512 pixel resolution and classify it using the random forest model. The regions were assumed to be contiguous, so we configured the system to classify only the fourth pixel in each direction and interpolate between these locations. This sacrificed some spatial resolution but improved speed by a factor of 16. Next, the onboard procedure would identify connected regions and apply a distance transform to find each region's center point. Finally, it would produce a thumbnail image fragment around each center point that was associated with a region above an area threshold. This provided a concise summary of image content. At the same time, it recorded both the classification map and an even more parsimonious text file containing the area and class label of each region. These could be used for summary downlink packages or as a cue for automated prioritization.

Although both power and radiation concerns impacted the onboard planning and execution software, neither was a significant factor for the classifier algorithm. The processor was not throttled, so the power cost of each algorithm was directly proportional to processing time. Nor was the algorithms' radiation tolerance significant; it is notoriously difficult to predict the effects of degradation and single-event upsets, which depend highly on the orbit and the self-shielding provided by spacecraft structures. Instead, we relied on the existing two-tier spacecraft fault protection. There was no strict constraint on the processing time per image, but we set an informal goal for processing to take just a handful of minutes or less per image. This guaranteed that the classifier could keep pace with the rate of data acquisition and that nearly any image we wished to try and downlink would in principle have classified products available.

IPEX was launched into low Earth orbit on an Atlas V rocket from Vandenberg Air Force Base on December 5, 2013, with the pretrained random forest model and procedure onboard (Figure 14 Left). Within several days, orbital parameters had been determined with sufficient precision for accurate ground-relative tracking and occasional sporadic contacts. The IPEX orbit was highly elliptical, and the spacecraft came near 400 km altitude at perigee. Checkout procedures lasted for approximately 1 month; during this time, the CASPER system began acquiring and processing images on both Atmel and Gumstix processors. After several more weeks, the first classification maps were downlinked via a transient connection to an amateur radio band receiver at CalPoly San Luis Obispo. This provided the first results of the field experiment.

6.3. Results

At the time of this writing, IPEX had processed more than 5,000 images using the classifier. Highly compressed thumbnail versions of several hundred RGB images have been downlinked through conventional scheduling strategies. The majority of these visible images were not interpretable due to data corruption during transmission. Minor connection interruptions were common and caused most downlinks to fail midway. Of the images that have been successfully transmitted, some are associated with the telemetry for us to evaluate the classification. The simplest and most common telemetry package is ASCII output providing algorithm runtimes and the centroids and areas of the connected regions. We reviewed the downlinked data and were able to identify 12 distinct downlinked images with this basic information. It soon became apparent that the mission would

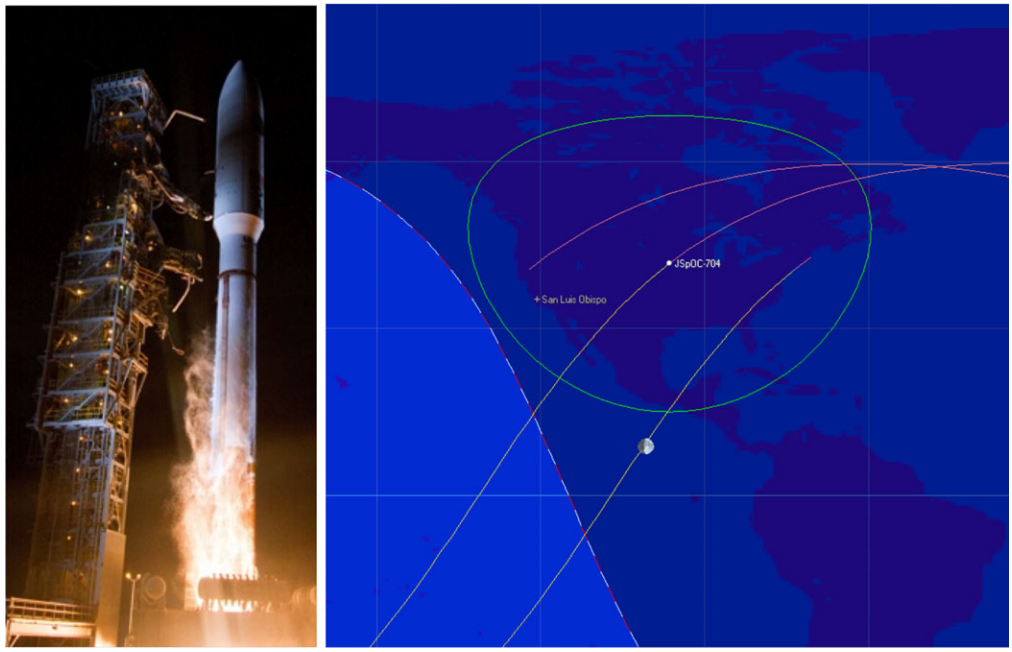

Figure 14. Left: the IPEX launch at Vandenberg Air Force Base (image credit: United Launch Alliance). Deployment occurred several hours after launch. Right: an IPEX orbital track for approximately 3 hours (two orbits) showing the location of the ground station and the spacecraft contact cone.

only be able to downlink a handful of frames. We subsequently began a schedule of manual downlink in which we identified promising good-quality image candidates and commanded prioritized transmission of these classifier results. In this manner, we were eventually able to obtain full classification maps for five well-illuminated, noncorrupted scenes.

The classifier downlink packages varied in their completeness because of the variability in the downlink process. We always evaluated the classifier predictions using the best information available. In the best cases, the original image and onboard classification result were both available. We have included the five classification maps in this paper (Figure 15). When the classification maps were not available, we compared the transmitted class centroid locations against the original thumbnail image to assess the classification quality. Specifically, we selected the center of the largest surface region as a target point to evaluate whether it would have fallen on a clear sky pixel. When no clear sky was present in the scene, either because the scene was totally clouded or the image contained only outer space, we recorded the system as having given the correct answer only when it did not identify any clear sky pixels. Otherwise, the target was judged to be correct if it lay on cloud-free land or ocean.

Table II shows the result for the entire data set. The columns show a unique image identifier and a "reason" field indicating whether the image was downlinked by conventional scheduling or a special request. Other columns show the direction the camera was pointed; the presence of space, cloud, and surface classes; the runtime in seconds; the quality of the target decision; the presence of a downlinked classification map; and a subjective assessment of the classification quality based on class centroids, or when available, the classification map. Overall, performance was promising. As predicted, the classifier had no trouble distinguishing the worst-quality images, such as black zenith-pointed images of outer space. Nadir- and horizon-pointed images under good illumination conditions generally showed accurate classifications. In other words, nearly all of the selected centroid points were "correct" and would have resulted in an appropriate targeting or abstention decision. This represents a significant improvement vis-à-vis status quo sampling and is consistent with the ISS simulations.

We note that, while these initial results indicate the classifier was functioning reasonably, the pixel maps to be transmitted were selected because the compressed returned images on the ground predicted that the scene geometry, content, and illumination would be reasonable and that the classification result was worth downlinking for analysis. Most failure cases involved imaging conditions for which the classifier was not trained, such as direct sun-blinding or poorly illuminated scenes near the night/day terminator. We have noted these cases in the table. Another minor failure condition, which appears in the classification map for image `cebdf`, is an occasional confusion between clouds

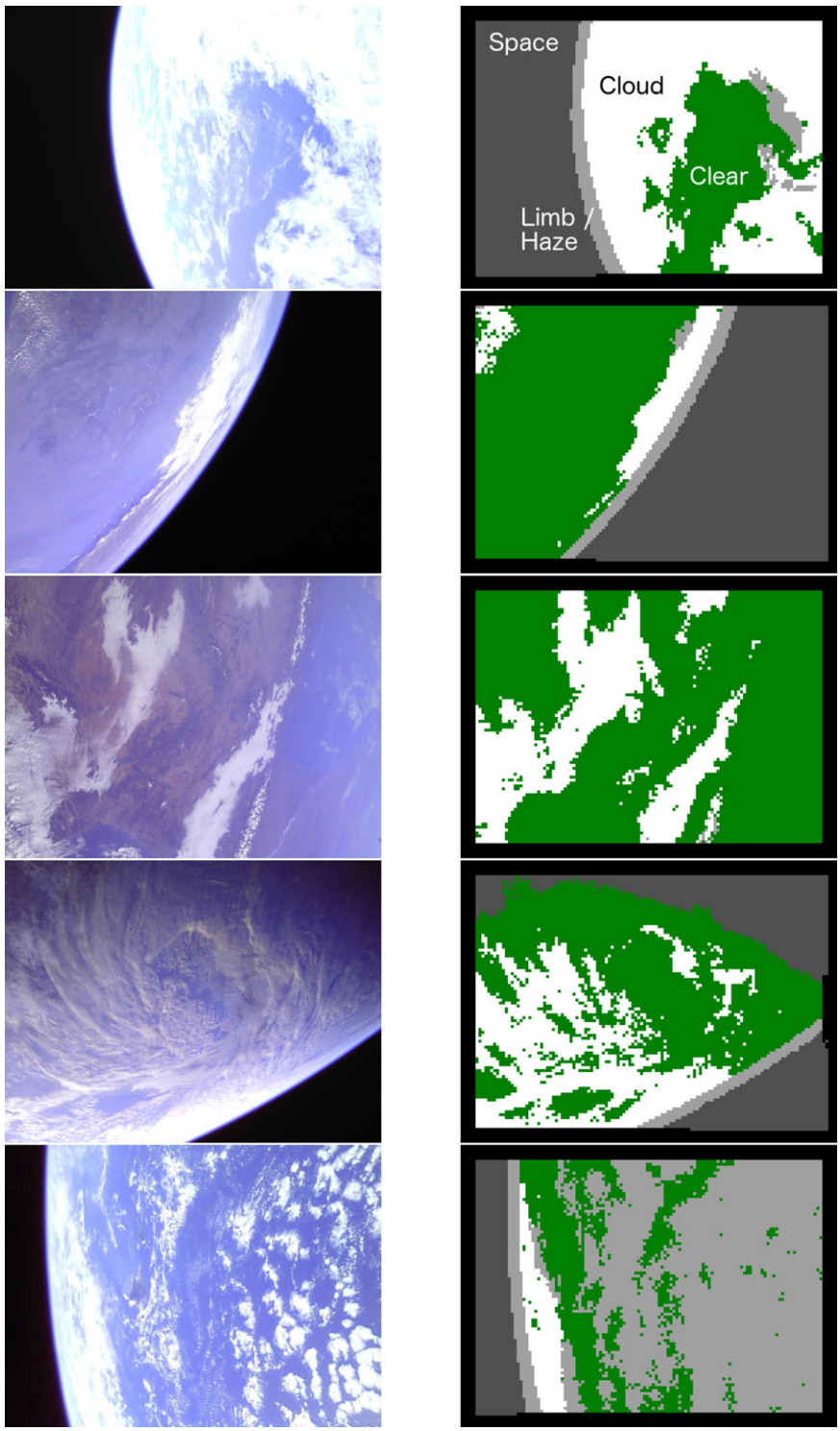

Figure 15. All returned IPEX classification maps. Top to bottom: 3746b, 866a3, ae0d6, 574ce, cebdf. Color coding: Black: unclassified. Dark gray: outer space. Light gray: horizon/haze. White: opaque cloud. Green: clear.

Table II. Performance for downlinked IPEX images. Notes signify: [1] Confusion between haze and cloud classes. [2] Camera blinded by sun glint. [3] Poor illumination during imaging of the terminator.

Image	Reason	Pointing	Space	Cloud	Surface	Runtime	Target	Class map	Skill	Notes
cebdf	random	horizon	x	x	x	17 s	good	x	fair	1
4b146	random	horizon	x			21	poor		poor	2
4d0b7	random	zenith	x			9	good		good	
56d35	random	nadir		x	x	22	good		fair	1
574ce	random	nadir		x	x	40	poor	x	fair	3
5f5c2	random	zenith	x			30	poor		poor	2
69821	random	zenith	x			21	good		good	
3746b	manual	horizon	x	x	x	118	good	x	good	
67b0c	manual	horizon	x	x	x	36	good		good	
866a3	manual	horizon	x	x	x	48	good	x	good	
ae0d6	manual	nadir		x	x	29	good	x	good	
df192	manual	horizon	x		x	110	good		good	

and haze classes. This is because the limitations of the training set, which was acquired in a balloon flight over land: sharp-bordered clouds over very dark ocean showed higher color contrast than any feature in the training set except the horizon. Thus, when flying in space, the classifier sometimes selects the horizon/haze class for these high-contrast ocean areas. This could easily be addressed by merging the horizon/haze and clouds classes into a single category. Alternatively, if bandwidth permitted, one could uplink a new random forest model trained on a more representative data set. Despite this fact, the initial results demonstrate that the classifier had made reasonable classification decisions in the majority of cases.

7. DISCUSSION

This work provides several important lessons for future missions. First, the ISS study shows that cloud screening can produce significant performance gains for targeted instruments, increasing their yield by a factor of 2 with trivial increases in mass and power requirements. Provided images are well conditioned (e.g., nadir-pointed, with the sun well above the horizon) even a simple classifier can achieve significant improvements. Because all missions that observe Earth in visible wavelengths must deal with pervasive clouds, the applicability is potentially quite broad.

A second lesson is the importance of illumination differences and sun glint. One could account for these factors in preprocessing when the local solar ephemeris is known, as would be the case for most or all Earth-orbiting spacecraft. One could simply normalize by the cosine of the solar zenith angle as in (Thompson et al., 2014a). It will also be important to handle sun glint cases, either by excluding certain images based on a known spacecraft pose or by filtering artifacts with simple heuristic rules or a dedicated sun-glint classifier. Neither of these measures was anticipated for IPEX, but despite this fact, the system still returns good results for most of the images we have observed.

Third, a more representative training set would improve the performance further. We suggest that it is important to reserve time and resources in the initial operations phase of a mission for this "calibration" step to achieve maximum performance during the main mission. High-altitude balloon flights provide a compromise, but imperfect, substitute for realistic training images collected on-orbit.

Above all, this work has achieved several firsts: the first use artificial intelligence or computer vision onboard a small spacecraft, and the first image-based cloud screening onboard any spacecraft. It also introduced random forests to spaceflight, with real-time scene understanding to inform adaptive data collection, processing, and downlink. The experiments pave the way for future use onboard instruments such as OCO-3. It is slated to be used in the INSPIRE interplanetary spacecraft (Klesh et al., 2013) for finding and subframing planetary objects during the cruise phase of the mission.

ACKNOWLEDGMENTS

Random forest code similar to the ground version of the classifier is now available online (Thompson et al., 2014b). Portions of this work were performed by the Jet Propulsion Laboratory, California Institute of Technology, under contract with the National Aeronautics and Space Administration (NASA). The IPEX project was funded by NASA's Earth Science Technology Office. The OCO-3 Project is part of the Earth System Science Pathfinder (ESSP) Program directed by the program director of the NASA Earth Science Division (ESD). Contributors to the TextureCam codebase include Dmitriy Bekker, Brina Bue, Daniel Howarth, Kevin Ortega, and Greydon Foil. The TextureCam project is supported by the NASA Astrobiology Science and Technology Instrument

Development Program (NNH10ZDA001N-ASTID). We thank Susan Runco and the HDEV team for their help acquiring and using this imagery.

REFERENCES

Ackerman, S., Holz, R., Frey, R., Eloranta, E., Maddux, B., & McGill, M. (2008). Cloud detection with MODIS. Part II: validation. Journal of Atmospheric and Oceanic Technology, 25(7), 1073–1086.

Ackerman, S. A., Strabala, K. I., Menzel, W. P., Frey, R. A., Moeller, C. C., & Gumley, L. E. (1998). Discriminating clear sky from clouds with MODIS. Journal of Geophysical Research: Atmospheres, 103(D24), 32141–32157.

Bekker, D. L., Thompson, D. R., Abbey, W. J., Cabrol, N. A., Francis, R., Manatt, K. S., Ortega, K. F., & Wagstaff, K. L. (2014). Field demonstration of an instrument performing automatic classification of geologic surfaces. Astrobiology.

Brakenridge, G. R., & Anderson, E. (2006). MODIS-based flood detection, mapping and measurement: The potential for operational hydrological applications. In Marsalek, J., Stancalie, G., and Balint, G., editors, Transboundary Floods: Reducing Risks Through Flood Management, volume 72 of NATO Science Series: IV: Earth and Environmental Sciences, pages 1–12. Springer, Netherlands.

Breiman, L. (2001). Random forests. Machine learning, 45(1), 5–32.

Carroll, M., Townshend, J., DiMiceli, C., Noojipady, P., & Sohlberg, R. (2009). A new global raster water mask at 250 meter resolution. International Journal of Digital Earth.

Cazorla, A., Olmo, F. J., & Alados-Arboledas, L. (2008). Development of a sky imager for cloud cover assessment. J. Opt. Soc. Am. A, 25(1), 29–39.

Chien, S., Doubleday, J., Tran, D., Thompson, D., Wagstaff, K., Bellardo, J., Francis, C., Baumgarten, E., Williams, A., Yee, E., Fluitt, D., Stanton, E., & Piug-Suari, J. (2014). Onboard autonomy on the intelligent payload experiment (IPEX) Cubesat Mission as a pathfinder for the proposed HyspIRI Mission intelligent payload module. International Symposium on Artificial Intelligence, Robotics, and Automation for Space (i-SAIRAS), Montreal, Canada.

Chien, S., Knight, R., Stechert, A., Sherwood, R., & Rabideau, G. (2000). Using iterative repair to improve the responsiveness of planning and scheduling. Proceedings of the Fifth International Conference on Artificial Intelligence Planning and Scheduling, pages 300–307.

Chien, S., McLaren, D., Rabideau, G., Silverman, D., Mandl, D., & Hengemihle, J. (2010). Onboard processing for low-latency science for the HyspIRI Mission. International Symposium on Space Artificial Intelligence, Robotics, and Automation for Space (i-SAIRAS), Sapporo, Japan.

Chien, S., McLaren, D., Tran, D., Davies, A., Doubleday, J., & Mandl, D. (2013). Onboard product generation on earth observing one: A pathfinder for the proposed HyspIRI Mission intelligent payload module. Selected Topics in Applied Earth Observations and Remote Sensing, IEEE Journal of, 6(2), 257–264.

Chien, S., Silverman, D., Davies, A., & Mandl, D. (2009). Onboard science processing concepts for the HyspIRI Mission. Intelligent Systems, IEEE, 24(6), 12–19.

DigitalGlobe (2014). http://worldview3.digitalglobe.com.

Doggett, T., Greeley, R., Chien, S., Cichy, B., Davies, A., Rabideau, G., Sherwood, R., Tran, D., Baker, V., Dohm, J., & Ip, F. (2006). Autonomous on-board detection of cryospheric change. Remote Sensing of Environment, 101, 447–462.

Eastman, R., Warren, S., & Hahn, C. (2011). Variations in cloud cover and cloud types over the ocean from surface observations, 1954-2008. Journal of Climate, 24(22), 5914–5934.

Eldering, A., Kaki, S., Crisp, D., & Gunson, M. R. (2013). The OCO-3 Mission. American Geophysical Union, Fall Meeting, pages #A21G–0134.

Estlin, T. A., Bornstein, B. J., Gaines, D. M., Anderson, R. C., Thompson, D. R., Burl, M., Casta no, R., & Judd, M. (2012). AEGIS automated science targeting for the mer opportunity rover. ACM Trans. Intell. Syst. Technol., 3(3), 50:1–50:19.

Foil, G., Thompson, D. R., Abbey, W., & Wettergreen, D. (2013). Probabilistic surface classification for rover instrument targeting. IEEE/RSJ International Conference on Intelligent Robots and Systems.

Frey, R. A., Ackerman, S. A., Liu, Y., Strabala, K. I., Zhang, H., Key, J. R., & Wang, X. (2008). Cloud detection with MODIS. Part I: Improvements in the MODIS cloud mask for collection 5. Journal of Atmospheric and Oceanic Technology, 25(7), 1057–1072.

Fuchs, T. J., Bue, B. D., Castillo-Rogez, J., Wagstaff, K., & Thompson, D. R. (2014). Autonomous onboard surface feature detection for flyby missions. Proceedings of the International Symposium on Artificial Intelligence, Robotics and Automation in Space, Montreal, Canada.

Gómez-Chova, L., Camps-Valls, G., Calpe-Maravilla, J., Guanter, L., & Moreno, J. (2007). Cloud-screening algorithm for ENVISAT/MERIS multispectral images. IEEE Transactions on Geoscience and Remote Sensing, 45(12), 4105–4118.

Griffin, M. K., Burke, H. K., Mandl, D., & Miller, J. (2003). Cloud cover detection algorithm for EO-1 hyperion imagery. 17th SPIE AeroSense 2003, Orlando FL, Conference on Algorithms and Technologies for Multispectral, Hyperspectral and Ultraspectral Imagery IX, pages 483–494.

Han, Y., Kim, B., Kim, Y., & Lee, W. H. (2014). Automatic cloud detection for high spatial resolution multi-temporal images. Remote Sensing Letters, 5(7), 601–608.

Ip, F., Dohm, J., Baker, V., Doggett, T., Davies, A., no, R. C., Chien, S., Cichy, B., Greeley, R., Sherwood, R., Tran, D., & Rabideau, G. (2006). Flood detection and monitoring with the Autonomous Sciencecraft Experiment onboard EO-1. Remote Sensing of Environment, 101(4), 463–481.

King, M., Platnick, S., Menzel, W., Ackerman, S., & Hubanks, P. (2013). Spatial and temporal distribution of clouds observed by MODIS onboard the Terra and Aqua satellites. IEEE Transactions on Geoscience and Remote Sensing, 51(7), 3826–3852.

Klesh, A. T., Baker, J. D., Bellardo, J., Castillo-Rogez, J., Cutler, J., Halatek, L., Lightsey, E. G., Murphy, N., & Raymond, C. (2013). INSPIRE: Interplanetary NanoSpacecraft Pathfinder in Relevant Environment. American Institute of Aeronautics and Astronautics.

Lee, J., Weger, R., Sengupta, S., & Welch, R. (1990). A neural network approach to cloud classification. IEEE Transactions on Geoscience and Remote Sensing, 28(5), 846–855.

Liu, S., Zhang, L., Zhang, Z., Wang, C., & Xiao, B. (2015). Automatic cloud detection for all-sky images using superpixel segmentation. Geoscience and Remote Sensing Letters, IEEE, 12(2), 354–358.

Martins, J. V., Tanré, D., Remer, L., Kaufman, Y., Mattoo, S., & Levy, R. (2002). MODIS cloud screening for remote sensing of aerosols over oceans using spatial variability. Geophysical Research Letters, 29(12), 8009.

Mercury, M., Green, R., Hook, S., Oaida, B., Wu, W., Gunderson, A., & Chodas, M. (2012). Global cloud cover for assessment of optical satellite observation opportunities: A HyspIRI case study. Remote Sensing of Environment, 126(0), 62–71.

Minnis, P., Trepte, Q. Z., Sun-Mack, S., Chen, Y., Doelling, D. R., Young, D. F., Spangenberg, D. A., Miller, W. F., Wielicki, B. A., Brown, R. R., et al. (2008). Cloud detection in nonpolar regions for CERES using TRMM VIRS and terra and aqua MODIS data. IEEE Transactions on Geoscience and Remote Sensing, 46(11), 3857–3884.

Murtagh, F., Barreto, D., & Marcello, J. (2003). Decision boundaries using Bayes factors: the case of cloud masks. IEEE Transactions on Geoscience and Remote Sensing, 41(12), 2952–2958.

Otsu, N. (1979). A threshold selection method from gray-level histograms. Systems, Man and Cybernetics, IEEE Transactions on, 9(1), 62–66.

Perkins, T., Adler-Golden, S., Matthew, M. W., Berk, A., Bernstein, L. S., Lee, J., & Fox, M. (2012). Speed and accuracy improvements in FLAASH atmospheric correction of hyperspectral imagery. Optical Engineering, 51(11), 111707–1.

Runco, S. (2014). Private correspondence. HDEV data available from http://www.nasa.gov/mission_pages/station/research/experiments/917.html.

Shotton, J., Johnson, M., & Cipolla, R. (2008). Semantic texton forests for image categorization and segmentation. In Computer Vision and Pattern Recognition, 2008. CVPR 2008. IEEE Conference on, pages 1–8. IEEE.

Swartwout, M. (2014). The First One Hundred CubeSats: A Statistical Look. JoSS, 2, 213–233.

Taylor, T., O'Dell, C., O'Brien, D., Kikuchi, N., Yokota, T., Nakajima, T., Ishida, H., Crisp, D., & Nakajima, T. (2012). Comparison of cloud-screening methods applied to GOSAT near-infrared spectra. IEEE Transactions on Geoscience and Remote Sensing, 50(1), 295–309.

Thompson, D., Bornstein, B., Chien, S., Schaffer, S., Tran, D., Bue, B., Castano, R., Gleeson, D., & Noell, A. (2013). Autonomous spectral discovery and mapping onboard the EO-1 spacecraft. IEEE Transactions on Geoscience and Remote Sensing, 51(6), 3567–3579.

Thompson, D., Green, R., Keymeulen, D., Lundeen, S., Mouradi, Y., Nunes, D., Castano, R., & Chien, S. (2014a). Rapid spectral cloud screening onboard aircraft and spacecraft. Geoscience and Remote Sensing, IEEE Transactions on, 52(11), 6779–6792.

Thompson, D. R., Abbey, W., Allwood, A., Bekker, D., Bornstein, B., Cabrol, N. A., no, R. C., Estlin, T., Fuchs, T., & Wagstaff, K. L. (2012). Smart cameras for remote science survey. Proceedings of the International Symposium on Artificial Intelligence, Robotics and Automation in Space, Turin Italy.

Thompson, D. R., Bekker, D., Bue, B., Foil, G., Ortega, K., & Wagstaff, K. (2014b). Texturecam: Geologic image classification with random forests. https://github.com/davidraythompson/texturecam.

Thompson, D. R., Gao, B.-C., Green, R. O., Roberts, D. A., Dennison, P. E., & Lundeen, S. (2015). Atmospheric Correction for Global Mapping Spectroscopy: ATREM Advances for the HyspIRI Preparatory Campaign. Remote Sensing of Environment, 167, 64–77.

Ustream (2014). http://www.ustream.tv/recorded/51601838.

Wagstaff, K., Thompson, D., Abbey, W., Allwood, A., Bekker, D., Cabrol, N., Fuchs, T., & Ortega, K. (2013). Smart, texture-sensitive instrument classification for in situ rock and layer analysis. Geophysical Research Letters, 40(16), 4188–4193.

Wang, M., & Shi, W. (2006). Cloud masking for ocean color data processing in the coastal regions. IEEE Transactions on Geoscience and Remote Sensing, 44(11), 3196–3105.

Wettergreen, D., Foil, G., Furlong, M., & Thompson, D. R. (2014). Science autonomy for rover subsurface exploration of the Atacama Desert. AI Magazine (in press).

Wie, B., Bailey, D., & Heiberg, C. (2002). Rapid multitarget acquisition and pointing control of agile spacecraft. Journal of Guidance, Control, and Dynamics, 25(1), 96–104.

Zhang, Q., & Xiao, C. (2014). Cloud detection of RGB color aerial photographs by progressive refinement scheme. Geoscience and Remote Sensing, IEEE Transactions on, 52(11), 7264–7275.

Zhu, Z., & Woodcock, C. E. (2014). Automated cloud, cloud shadow, and snow detection in multitemporal landsat data: An algorithm designed specifically for monitoring land cover change. Remote Sensing of Environment, 152(0), 217–234.